Memory Organization and Cache Replacement Policies

Divvela Rakesh - CS23BTECH11017
Madishetty Adityanath-CS23BTECH11032
Rongali Mohit naidu-CS23BTECH11051
Nerusupalli RamPanikar Reddy-CS23BTECH11040

21 October 2024

Contents

1 Recap **1**

2 Memory address Illustration **2**

3 Example Illustration on how Cache works **3**

4 Main memory supporting Caches **4**

5 Content Addressable Memory (CAM) **7**

6 Replacement policy **9**

7 Exercises **10**

1 Recap

- **Memory Wall**
  This is a problem in computing where processor improvement **out-paces** the memory transfer time , so any improvement in processor seems insignificant. we need to tackle this by improving transfer time.

- **Memory Technologies**
  The main technologies currently used are:

  1. SRAM (static Random access memory)
  2. DRAM (Dynamic Random access memory)
  3. Magnetic Disks

There are trade-offs between cost and access time for the above technologies.The access time increases top to bottom but cost also decreases.

- **Locality behaviour of programs**

  1. **Temporal locality**
     The locality principle stating that if a data location is referenced then it will tend to be referenced again soon.

     "Past is Future"

  2. **Spatial locality**
     The locality principle stating that if a data location is referenced, data locations with nearby addresses will tend to be referenced soon

- **Briefing cache Recap**

  1. **Cache** is level of the memory hierarchy between the processor and main memory,which helps in improvement of access time for any kind of data.
  2. Memory address while placed in cache it is divided into various fields named
     (a) Tag field
     (b) Cache index
     (c) Offset bits

     using the above , various cache techniques can be implemented like Direct mapping , Associative caches
  3. **Write Policies**
     (a) write through or write back for HITS
     (b) write allocate or no write allocate for misses

2 Memory address Illustration

While memory is generally organized linearly, to simplify our understanding of caches in the following discussion, we'll group memory into blocks of 4 or 8 bytes (specifically 4 bytes in the figure below). Each block shares the same base address, differing only in the last few bits
For example:
If the processor requests memory at address **1101** and if it results in a cache miss, then with a cache block size of 4 bytes, the cache will load all memory addresses labeled as **11xx** from memory. If the cache block size were instead 8 bytes, then the cache would fetch all addresses differing only in the last 3 bits. To visualize this, we could extend the table, grouped into blocks of 8 addresses per row.

Base	Byte 0	Byte 1	Byte 2	Byte 3
11xx	1100	1101	1110	1111
10xx	1000	1001	1010	1011
01xx	0100	0101	0110	0111
00xx	0000	0001	0010	0011

Table 1: Memory Organization Table

3 Example Illustration on how Cache works

In this we take 8 blocks,1 word/block,2-Way set associative.

Index	V	Tag	Data
00	N		
01	N		
10	N		
11	N		
00	N		
01	N		
10	N		
11	N		

Table 2: Initial state

Index	V	Tag	Data
00	Y	100	Mem[10000]
01	N		
10	Y	100	Mem[10010]
11	Y	000	Mem[00011]
00	N		
01	Y	111	Mem[11101]
11	Y	101	Mem[10110]
11	N		

Table 3: after few insertions without conflict

Index	V	Tag	Data
00	N		
01	N		
10	N		
01	N		
11	N		
00	N		
10	Y	101	Mem[10110]
11	N		

Table 4: After one insertion

Index	V	Tag	Data
00	Y	100	Mem[10000]
01	N		
10	y	100	Mem[10010]
11	y	000	Mem[00011]
00	y	000	Mem[00000]
01	y	111	Mem[11101]
10	y	101	Mem[10110]
11	N		

Table 5: Finally after inserting conflicted element

- Initial state table represents the initial state of Cache memory.
- Next we want to access :

Mem addr	Binary addr	Hit/Miss	Set
22	101 10	MISS	10

Table 6: 1st entry

Its a miss so we need to insert in appropriate place in cache.
The above data has 10_2 as index (last two bits) so the tag becomes 101_2 . initially as the index 10_2 has **2 empty places** we can place the data in any of the one place

- after acessing few values and indices 00 10 11 01 11 are occupied
- Now we want to access

Mem addr	Binary addr	Hit/Miss	Set
0	000 00	MISS	00

Table 7: 00 conflicting

It's tag is absent in the cache memory so its a miss.Now the index for this is 00 so we have two choices for this address.Now we have two options either fill it in a place where Valid bit is "N" or replace it with already occupied 00 block .

- Its better to place this in the block with Valid bit as "N" so that we can access both memory locations from cache
- We discuss more about the placement of data when we get conflicts in the subsequent slides.
- Now if we want access:

Mem addr	Binary addr	Hit/Miss	Set
16	100 00	HIT	00

Table 8: cache hit

Now at index 00_2 we can find element whose tag matches with the required one (which we want to access) , so we can directly use the data from cache memory using the byte offset that is present in address to find the correct element among the data that set 00 holds , instead of fetching it from main memory.

4 Main memory supporting Caches

- We generally use **DRAM** for main memory because of it's cost effectiveness and even though it's slower than **SRAM** its reasonably faster for

some applications.

- This is also a volatile memory i.e it loses data after power of the system is turned off.
- The CPU and DRAM are connected via a bus, The CPU communicates with DRAM (main memory) through this bus to read and write data when needed (like cache miss).
- Bus clock is typically slower than CPU clock . The CPU generates its own clock rate because it operates at a high speed.
- Width of bus is fixed ,i.e data it can transfer per cycle is fixed
- Let us consider an example where we want to read 4-words from memory .Here in this memory organization each block of memory is one word wide.(i.e single access cycle one word can be transfered)

  information given :

  1. consider 1 bus cycle for address transfer
  2. 15 bus cycles per DRAM access
  3. 1 Bus cycle per data transfer

  Now Let's Calculate the **Miss penalty** for this arrangement.

  1. for address transfer **one bus** cycle.
  2. 15 bus cycles per DRAM access and we need to access 4 times
  3. for address transfer of each word is 1 bus cycle for 4 words it is 4*1 bus cycles.

$$\text{Total} = 1 + 4 * 15 + 4 * 1 = 65 \text{ bus cycles}$$

  Now corresponding bandwidth for this becomes 4*4bytes /65 bus cycles which is **0.25Bytes/cycle**.

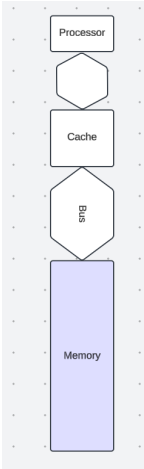


Figure 1: One-word wide memory organization

- 4-WORD WIDE MEMORY
  Width of the Bus is increased so now 4 words can be accessed simultaneously and can be transferred simultaneously.

  $$\text{Miss-penalty} = 1 + 15 + 1 = 17 \text{ bus cycles}$$

  Now corresponding bandwidth for this becomes 4*4 bytes /17 bus cycles which is **0.94 Bytes/cycle**.

  refer fig

- **Memory is divided into 4 banks and width of the bus is one word**:
  This decreases access time because now once address is transferred all 4 words are accessed simultaneously (each word from one bank , memory is divided in such way) so access time is just 15 bus cycles .But transfer happens one after the other.

  $$\text{Miss penalty} = 1 + 15 + 4 * 1 = 20\text{bus cycles}$$

  Now corresponding **bandwidth** for this becomes 4*4 bytes /20 bus cycles which is **0.8 Bytes/cycle**.

- **Observation** : By dividing memory into 4 banks , we can decrease the miss penalty also without making hardware complex by increasing bus size.

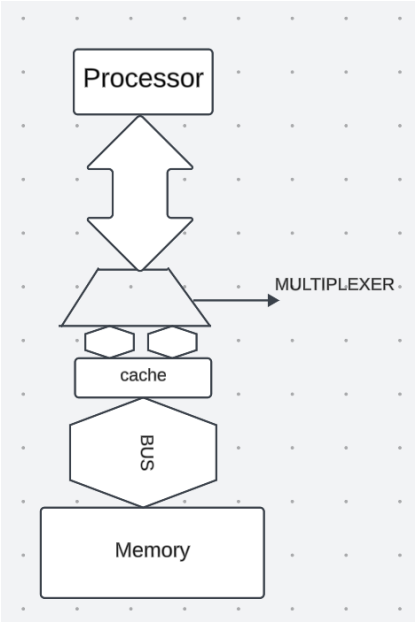


Figure 2: 4-word wide memory organization

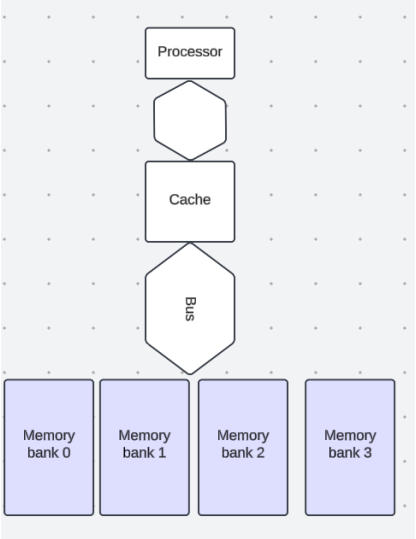


Figure 3: Memory is divided into 4 banks

5 Content Addressable Memory (CAM)

- Generally a memory unit receives index or address to fetch the data.But In set associative or fully associative cache, we have to check whether the input (i.e., tag) matches the data in the location as there are multiple options for a data to go. Such kind of memories are called content addressable memory or CAM.
- So the Hardware becomes complex for set associative caches as we need extra comparators for checking.
- We also need an additional multiplexor.

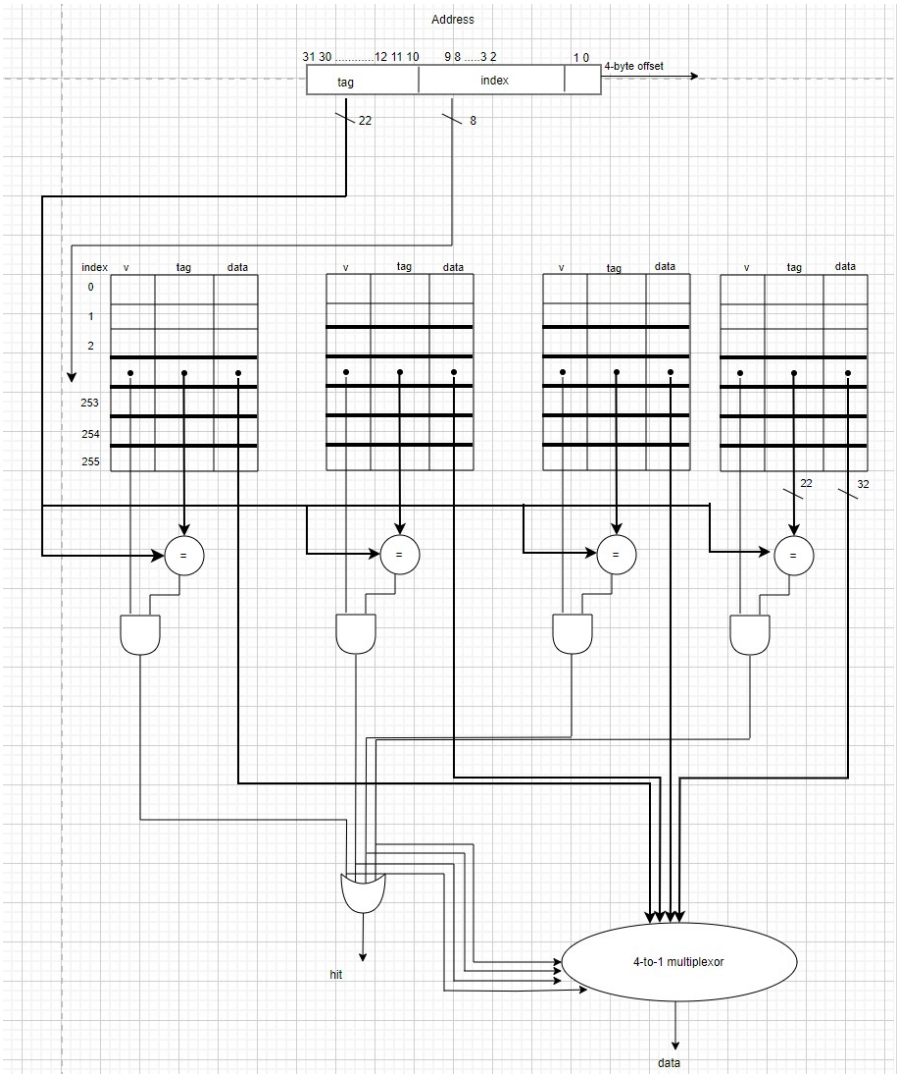


Figure 4: Above is the example for 4 way associativity cache

6 Replacement policy

- In case of conflict of matching of index during the Accessing of data.We can follow some policies to maximize the efficiency of the cache. **This means increasing hit rate and decreasing miss rate.**
- In direct map cache we have no choice but to replace the older data with the recently accessed data since an address has fixed place to go and each index can store only one block.
- **In set associative cache**
  1. If there is an invalid entry(valid bit="N") in a set that is to be used to fill the element, we use that since it allows us to store both pieces of data.
  2. If there is no invalid entry we have to replace the any one of the entries. **this leads to some replacement policies.**
- **FIFO**(FIRST IN FIRST OUT)
  1. In this policy the eviction is done on the basis of **arrival of data**.i.e block which has arrived first will get replaced.
  2. For this we have to maintain the timestamps at which the blocks have arrived.(which is basically an another overhead).
  3. It is advantageous in sequential access.
  4. It is disadvantageous if the program has temporal locality and random access patterns.
  5. Overall generally it has low hit rate. so its usage is low and is specific.
- **LRU**(Least Recently Used)
  1. In this policy the eviction is done the basis of **data which is unused for the longest time.**
  2. It is more complex implementation than previous one and has more overheads than previous implementation. We need to maintain the order of access of the ways for each set.
  3. It is advantageous for programs having temporal locality (i.e where we often use the same piece of data much).
- **LFU**(Least Frequently Used)
  1. In this policy the eviction is done on the basis of **data which is used least number of times**.
  2. We need to maintain a counter for this implementation and counter with lowest value is evicted (overhead is a counter).
  3. In case of conflict for the above we can remove any one of the block (i.e equal frequencies).

4. There is issue when the newly arrived has temporal locality but it will have less count so it gets evicted always. In this scenario it is very disadvantageous. Hence, this policy is used together with some other policy.

- **Random**

1. In this policy, eviction is done randomly from available blocks from a set.
2. No overheads are required for this implementation.
3. We can't say anything about performance for this implementation as it is completely random.
4. For high associtivity Random works like LRU.
5. We can implement this using pseudo random number generators.

- We use combination of policies to achieve the max efficiency like

1. LRU+LFU
2. LRU+Random

- In general, there are policies which prioritize non-dirty blocks for eviction so that there would be no need to write back dirty blocks to the memory and thus miss penalty can be lower.

7 Exercises

- **Problem-1**
For a direct-mapped cache design with a 64-bit address, the following bits of the address are used to access the cache.

Tag	Index	offset
63-10	9-5	4-0

1. What is the cache block size (in words)?
sol: No of bits in bit offset is 5 bits so No of bytes in a cache block is 2^5 bytes which is 8 words.
2. How many blocks does the cache have?
sol: Index bits are from 9-5 which means it has 5 bits ,No of cache blocks is equals to 2^5 which is 32 .
3. What is the ratio between total bits required for such a cache implementation over the data storage bits?
sol: Total data storage=32 cache blocks=32*32 bytes.
Overheads In cache are Valid bit,Dirty bit and Tag.

Overhead storage=32*(No.of Tag bits+valid bit+Dirty bit)
=32*(54+1+1)=1792 bits=224 bytes.
Ratio=(Data storage+Overhead)/Data storage
=(224+1024)/1024
=1.21

Address												
Hex	00	04	10	84	E8	A0	400	1E	8C	CIC	B4	884
Dec	0	4	16	132	232	160	1024	30	140	3100	180	2180

4. For each reference, list (1) its tag, index, and offset, (2) whether it is a hit or a miss, and (3) which bytes were replaced (if any)?
   sol: Below is the table which list the things.
   Each block in a cache index has 32 bytes (4 words) so 0x00 to 0x1f

Byte Address	**Binary Address**	**Tag**	**Index**	**Offset**	**Hit /Miss /Replaced**	**Bytes Replaced**
0x00	0000 0000 0000	0x0	0x00	0x00	M	
0x04	0000 0000 0100	0x0	0x00	0x04	H	
0x10	0000 0001 0000	0x0	0000	0x10	H	
0x84	0000 1000 0100	0x0	0x04	0x04	M	
0xe8	0000 1110 1000	0x0	0x07	0x08	M	
0xa0	0000 1010 0000	0x0	0x05	0x00	M	
0x400	0100 0000 0000	0x1	0x00	0x00	M	0x00-0x1F
0x1e	0000 0001 1110	0x0	0x00	0x1e	M	0x400-0x41F
0x8c	0000 1000 1100	0x0	0x04	0x0c	H	
0c1c	1100 0001 1100	0x3	0x00	0x1c	M	0x00-0x1F
0xb4	0000 1011 0100	0x0	0x05	0x14	H	
0x884	1000 1000 0100	0x2	0x04	0x04	M	0x80-0x9F

   are fetched if the 0x00 is a miss. The others are also fetched likewise.

5. What is the hit ratio?
   sol:

$$\text{Hit ratio} = \text{Hits/Hits+Misses}$$
$$\text{Hit ratio} = 4/12 = 0.333$$

- **Problem-2**
  Consider a cache with 64 blocks and block size of 16 bytes.To what block

number does address 1200 map?
sol: The cache has 64 blocks so its index is of 6 bits and size of block is 16 bytes so bit offset is 4 bits.
1200 in binary is 10010110000 so last 4 bits for bit offset and next 6 bits for index.
which is 001011 =11 which is 11th block.